

Design Report: Concurrent Task Dispatcher in Rust

Architecture

My project is a simulation of a concurrent task dispatcher. The main thread creates a fixed 1000 tasks at 20ms intervals. Each task is either IO or CPU. The tasks are sent through a channel into the manager queue. The manager thread receives tasks, stores them in a ready queue, checks worker availability, checks global CPU availability, and then dispatches tasks to a bounded pool of 8 worker threads.

The program also has a monitor thread. The monitor is independent from the scheduler and samples the current CPU resource usage and active worker count every 10ms. At the end of the simulation, the program prints total runtime, average CPU usage, average worker usage, wait time, turnaround time, and completed task counts.

Data Structures

The task model stores an id, task kind, arrival time, and duration. IO tasks simulate 200ms of work and use 10 percent CPU resource. CPU tasks also simulate 200ms of work but use 35 percent CPU resource.

The FIFO version uses one VecDeque queue. The optimized version uses two VecDeque queues: one for IO tasks and one for CPU tasks. VecDeque is used because the scheduler needs efficient push_back and pop_front operations.

Synchronization Strategy

The program uses Rust channels for most communication. The main thread sends tasks to the manager. The manager sends tasks to workers. Workers send completion information back to the manager. Workers also report when they become available.

The shared state is protected with Arc and Mutex. The global CPU usage is shared because the manager reserves CPU before dispatching a task, workers release CPU after finishing a task, and the monitor reads CPU usage every 10ms. The active worker count is also shared because workers update it and the monitor reads it.

Scheduling Policy

The FIFO scheduler is the baseline. It uses a single queue and tries to dispatch the front task first. This policy is easy to explain, but it can suffer from head-of-line blocking. For example, if the front task is a CPU task and there is not enough CPU resource left, IO tasks behind it also have to wait even if they could run.

The optimized scheduler separates IO and CPU tasks. It tries to keep workers busy while never passing 100 percent CPU usage. The useful combinations are 2 CPU and 3 IO tasks, 1 CPU and 6 IO tasks, or 0 CPU and 8 IO tasks. This makes better use of the 8 worker lanes because IO tasks can continue running while CPU-heavy tasks wait for enough CPU capacity.

Metrics and Experiments

The program runs FIFO and optimized scheduling for 70 percent IO / 30 percent CPU and 80 percent IO / 20 percent CPU. The main comparison is total runtime and average CPU usage. The program also records average active workers, average wait time, average turnaround time, max wait time, and the number of IO and

CPU tasks completed.

The expected behavior is that optimized scheduling should usually finish faster than FIFO. FIFO is simpler, but optimized scheduling reduces wasted worker time by not blocking behind CPU-heavy tasks when IO tasks could still run.

Bug and Lessons Learned

One bug I had to think about was CPU overbooking. If workers increased CPU usage after receiving a task, the manager could accidentally dispatch too many tasks before the CPU value changed. I fixed this by having the manager reserve CPU before sending a task to a worker. The worker releases the CPU after the simulated work is finished.

Another important lesson is that clean shutdown needs to be designed directly. The manager waits until all 1000 tasks are completed, then sends shutdown messages to every worker. This prevents workers from waiting forever.

Remaining Trade-offs

The optimized scheduler improves runtime, but it is more complicated than FIFO. It also makes fairness slightly harder because CPU tasks might wait while IO tasks are used to fill the system. The code reduces this by still scheduling CPU tasks when enough CPU capacity is available. Starvation is unlikely in this fixed workload, but in an infinite stream of IO tasks, CPU fairness would need stronger aging logic.